String Matching

1 Introduction

What is String Matching?

String matching is the problem of finding all occurrences of a **pattern** inside a **text**.
It is very important in:

- text editors and search tools,
- DNA sequence analysis,
- search engines and information retrieval,
- pattern detection in large data streams.

Basic Idea

Given a text T and a pattern P, we want to determine all positions where P appears exactly in T.

2 Formal Definition of the Problem

Suppose:

$$T[1:n] \quad \text{is the text of length } n$$

$$P[1:m] \quad \text{is the pattern of length } m \leq n$$

and all characters come from a finite alphabet Σ.

Occurrence of a Pattern

We say that pattern P occurs in text T with **shift** s if

$$0 \leq s \leq n - m$$

and

$$T[s+1:s+m] = P[1:m].$$

Equivalently,

$$T[s+j] = P[j] \quad \text{for } 1 \leq j \leq m.$$

Valid and Invalid Shift

- If the above equality holds, then s is called a **valid shift**.
- Otherwise, s is an **invalid shift**.

Thus, the **string-matching problem** is:

Find all shifts s such that the pattern P occurs in the text T.

3 Example

Let

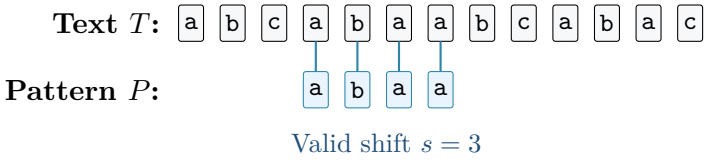
$$T = \texttt{abcabaabcabac}, \qquad P = \texttt{abaa}.$$

Then P occurs in T only once, at shift

$$s = 3.$$

That means the match starts at position

$$s + 1 = 4.$$



Valid shift $s = 3$

4 Algorithms Mentioned in the Chapter

Most efficient string-matching algorithms perform:

1. a **preprocessing phase** based on the pattern,
2. then a **matching phase** on the text.

Algorithm	Preprocessing Time	Matching Time
Naive	0	$O((n-m+1)m)$
Rabin–Karp	$\Theta(m)$	$O((n-m+1)m)$
Finite Automaton	$O(m\lvert\Sigma\rvert)$	$\Theta(n)$
Knuth–Morris–Pratt (KMP)	$\Theta(m)$	$\Theta(n)$

Key Observation

The naive algorithm does no preprocessing. The other algorithms preprocess the pattern first and then match faster.

5 Notation and Terminology

5.1 Set of All Strings

$$\Sigma^*$$

denotes the set of all finite-length strings formed from the alphabet Σ.

The empty string is denoted by

$$\varepsilon.$$

The length of a string x is denoted by

$$|x|.$$

5.2 Concatenation

If x and y are strings, then their concatenation is written as

$$xy,$$

which means the characters of x followed by the characters of y.

Also,

$$|xy| = |x| + |y|.$$

5.3 Prefix and Suffix

A string w is a **prefix** of a string x if

$$x = wy$$

for some string $y \in \Sigma^*$.

A string w is a **suffix** of a string x if

$$x = yw$$

for some string $y \in \Sigma^*$.

Examples

For the string `abcca`:

- `ab` is a prefix,
- `cca` is a suffix,
- ε is both a prefix and a suffix of every string.

Transitivity

Prefix and suffix relations are transitive:

- if a is a prefix of b and b is a prefix of c, then a is a prefix of c;
- if a is a suffix of b and b is a suffix of c, then a is a suffix of c.

6 Compact Prefix Notation

For convenience:

$$P_k = P[1 : k]$$

denotes the first k characters of the pattern.

So:

$$P_0 = \varepsilon, \qquad P_m = P.$$

Similarly:

$$T_k$$

denotes the first k characters of the text.

Using this notation, the string-matching problem can be restated as:

Compact Form

Find all shifts s such that

$$P \text{ is a suffix of } T_{s+m}.$$

Equivalently,

$$P \text{ matches the last } m \text{ characters of } T_{s+m}.$$

7 Overlapping-Suffix Lemma

Lemma 1 (Overlapping-Suffix Lemma)**.** *Suppose that x, y, and z are strings such that*

$$x \text{ is a suffix of } z \qquad \text{and} \qquad y \text{ is a suffix of } z.$$

Then:

- *if $|x| \leq |y|$, then x is a suffix of y;*
- *if $|x| \geq |y|$, then y is a suffix of x;*
- *if $|x| = |y|$, then $x = y$.*

Meaning of the Lemma

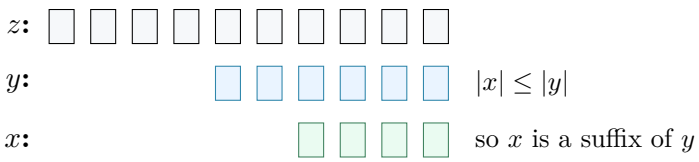
If two strings are both suffixes of the same larger string, then the shorter one must fit completely at the end of the longer one.
So two suffixes of the same string always **overlap in an ordered way**.

Intuition

Suppose both x and y end at the same final position of z.

Then:

- the shorter suffix lies entirely inside the longer suffix,
- if both have the same length, they must be exactly the same string.



8 Cost of Comparing Two Strings

In the discussion of algorithms, comparing two equal-length strings is treated as a primitive operation.

If two strings x and y are compared from left to right and comparison stops at the first mismatch, then the running time is proportional to:

$$\Theta(t+1),$$

where t is the number of initial matching characters.

Why $t+1$?

Even if the very first characters do not match, one comparison is still needed. So the cost is written as $\Theta(t+1)$ instead of $\Theta(t)$.

9 Final Summary

One-box Revision Summary

Text $T[1:n]$, pattern $P[1:m]$, $m \leq n$
A shift s is valid if $T[s+1:s+m] = P[1:m]$
Goal: find all valid shifts s, $0 \leq s \leq n-m$
Σ^* = set of all finite strings over Σ, ε = empty string
$P_k = P[1:k]$, $T_k = T[1:k]$
Prefix: $x = wy$, Suffix: $x = yw$
If two strings are suffixes of the same string, the shorter is a suffix of the longer
Major algorithms: Naive, Rabin–Karp, Finite Automaton, KMP